Homework on Bootstrapping

Leave your name and uni here

Problem 1: a randomized trial on an eye treatment

An ophthalmologist has designed a randomized clinical trial to assess the effectiveness of a new laser treatment (blue) against the traditional one (red). The response to be evaluated is visual acuity, which is measured by counting the number of letters correctly identified in a standard eye test. Out of the 40 patients who are eligible for laser treatment, 20 have only one suitable eye for the treatment and received one treatment allocated at random. The remaining 20 patients have both eyes suitable for laser treatment and received both treatments randomly assigned to the two eyes. Therefore, the study includes both paired comparison and two-sample data. The data are presented in the following R script, where the first 20 rows represent patients who received treatments in both eyes., and the last 10 rows represent patients who received treatment in only one eye.

```
> blue <- c(4,69,87,35,39,79,31,79,65,95,68,
           62,70,80,84,79,66,75,59,77,36,86,
           39,85,74,72,69,85,85,72)
> red <-c(62,80,82,83,0,81,28,69,48,90,63,
        77,0,55,83,85,54,72,58,68,88,83,78,
        30,58,45,78,64,87,65)
> acui<-data.frame(str=c(rep(0,20),
            rep(1,10)),red,blue)
```

Answer the following question:

(1) The treatment effect of the new laser treatment is defined as

$$\Delta = E(Y \mid \text{trt} = \text{new/blue}) - E(Y \mid \text{trt} = \text{traditional/red}).$$

Please construct a statistics from the collected data to estimate the treatment effect Δ.

(2) Propose a bootstrap algorithm to construct a 95% confidence interval for the treatment effect. Describe your bootstrap procedure. Based on the resulting confidence interval, can the new treatment effectively improve visual acuity?

(1)

```
blue <- c(4,69,87,35,39,79,31,79,65,95,68,
           62,70,80,84,79,66,75,59,77,36,86,
           39,85,74,72,69,85,85,72)
red <-c(62,80,82,83,0,81,28,69,48,90,63,
        77,0,55,83,85,54,72,58,68,88,83,78,
        30,58,45,78,64,87,65)
acui<-data.frame(str=c(rep(0,20),
            rep(1,10)),red,blue)
```

```r
delta_paired <- mean(acui$blue[1:20] - acui$red[1:20])
delta_indep <- mean(acui$blue[21:30]) - mean(acui$red[21:30])
delta_est <- (20 * delta_paired + 10 * delta_indep) / 30
```

The estimated treatment effect $\triangle$ is 3.0666667.

(2)

```r
set.seed(2)
bootstrap_delta_ci <- function(data, B = 1000, conf = 0.95) {
  n_paired <- 20
  n_unpaired <- 10

  delta_boot <- numeric(B)

  for (b in 1:B) {
    # Bootstrap sample of the paired data (first 20 rows)
    idx_paired <- sample(1:n_paired, n_paired, replace = TRUE)
    paired_blue <- data$blue[idx_paired]
    paired_red  <- data$red[idx_paired]
    delta_paired <- mean(paired_blue - paired_red)

    # Bootstrap sample of the unpaired data (last 10 rows)
    idx_indep <- sample(21:30, 10, replace = TRUE)

    blue_mean <- mean(data$blue[idx_indep])
    red_mean  <- mean(data$red[idx_indep])
    delta_indep <- blue_mean - red_mean

    # Combined estimate
    delta_boot[b] <- (20 * delta_paired + 10 * delta_indep) / 30
  }

  # Percentile CI
  alpha <- (1 - conf) / 2
  ci <- quantile(delta_boot, probs = c(alpha, 1 - alpha))

  return(list(
    estimate = mean(delta_boot),
    ci_lower = ci[1],
    ci_upper = ci[2],
    boot_dist = delta_boot
  ))
}
est_delta_ci <- bootstrap_delta_ci(acui)
est_delta_ci$ci_lower
```

```
##      2.5%
## -6.866667
```

```r
est_delta_ci$ci_upper
```

```
##    97.5%
## 12.86833
```

Procedure:

- Initialize confidence level, number of resamplings, and compared length of empty vector.
- We first resampling 20 paired patients with replacement and then compute the mean difference in visual acuity to obtain `delta_paired`. Then we resampling 10 unpaired/independent patients with replacement and then compute the difference between mean blue and red responses to obtain `delta_indep`. Then we use the results to get weighted treatment effect estimate for this bootstrap sample. Finally, we obtain the confidence interval(chosen confidence level = 95%) from the vector of estimations by taking the values from specific quantiles(2.5%, 97.5%).

Using above procedure, we obtain the confidence level of delta (-6.8666667, 12.8683333). 0 falls into this confidence interval, indicating that the difference between new and traditional treatments are not ststistically significant. Therefore, we conclude that this new treatment cannot effectively make improvement based on our results from bootstraping method.

Problem 2: Boston Housing Data

In this problem, you will explore the famous **Boston Housing** data, which was assembled in the 1970s as part of a research study aiming to understand how various factors, particularly environmental quality, impact housing prices. It draws heavily on the 1970 U.S. Census for Boston-area housing data and incorporates additional environmental and geographic indicators (e.g., pollution levels, proximity to the Charles River, highway access). The data is included in the MASS package in R, which can be loaded with

```
library(MASS)
data("Boston")
```

The data contains 506 observation and 14 variables, with each row corresponds to a census tract in the Boston area.

- crim: per capita crime rate by town
- zn: proportion of residential land zoned for lots over 25,000 sq.ft.
- indus: proportion of non-retail business acres per town
- chas: Charles River dummy variable (=1 if tract bounds river; 0 otherwise)
- nox: nitric oxides concentration (parts per 10 million)
- rm: average number of rooms per dwelling
- age: proportion of owner-occupied units built prior to 1940
- dis: weighted distances to five Boston employment centers
- rad: index of accessibility to radial highways
- tax: full-value property-tax rate per $10,000
- ptratio: pupil-teacher ratio by town
- black: 1000(Bk - 0.63)^2 where Bk is the proportion of Black residents by town
- lstat: percentage of lower status of the population
- medv: median value of owner-occupied homes in $1000s

2.1 contructing bootstrap confidence interval in quantile regression

Quantile regression is a modeling strategy that goes beyond predicting the average outcome. It models the conditional quantile of Y given the covariates X. For instance, you can use quantile regression to look at how the median (50th percentile), the 25th percentile, or the 75th percentile of housing prices change with

predictors. By doing so, you gain insights into how lower-priced homes vs. mid-range homes vs. higher-priced homes respond to changes in neighborhood and property characteristics.As we mentioned in the class, the asymptoic distribution of regression quantiles involves conditional densities, making direct Wald-type inference difficult. In the problem, you will fit quantile regression over *medv* with five predictors, *rm*, *lstat*, *chas*, *nox* and *tax* at quantile levels 0.1, 0.25, 0.5, 0.75 and 0.9.

In R code (using the quantreg package), you could write:

```
library(MASS)
data("Boston")
library(quantreg)

fit_qr = rq(medv ~ rm + lstat + chas + nox + tax, data = Boston, tau = c(0.1,0.25,0.5,0.75,0.9))
summary(fit_qr)
```

Problem 2.1.1

Fit quantile regression models for *medv* (median home value) in the Boston dataset at quantile levels,0.1, 0.25, 0.5, 0.75 and 0.8, using *rm* (average rooms), *lstat* (lower-status population),*chas*(Charles River), *nox* (pollution level), and *tax* (property tax rate) as predictors. Implement a paired bootstraping with $B = 1000$. For each replicate, re-estimate the coefficients at all five quantiles and store them. Afterward, for each coefficient at each quantile, construct and compare 95% percentile and basic bootstrap confidence intervals. Basedon your results, please discuss whether certain predictors have stronger effects on lower-priced homes versus higher-priced ones, noting any differences in significance or interval width across quantiles.

```
library(MASS)
data("Boston")
library(quantreg)
```

```
## Loading required package: SparseM
```

```
quantiles <- c(0.1, 0.25, 0.5, 0.75, 0.9)
B <- 1000
formula <- medv ~ rm + lstat + chas + nox + tax

boot_coefs <- array(NA, dim = c(B, length(quantiles), length(coef(rq(formula, tau = 0.5, data = Boston)))
                    dimnames = list(NULL, paste0("Q", quantiles), names(coef(rq(formula, tau = 0.5, data

# Run paired bootstrap
set.seed(2)  # for reproducibility
n <- nrow(Boston)

for (b in 1:B) {
  idx <- sample(1:n, n, replace = TRUE)
  boot_data <- Boston[idx, ]

  for (i in seq_along(quantiles)) {
    fit <- rq(formula, tau = quantiles[i], data = boot_data)
    boot_coefs[b, i, ] <- coef(fit)
  }
}

# Estimate original model (on full data)
orig_coefs <- sapply(quantiles, function(tau) coef(rq(formula, tau = tau, data = Boston)))

# Compute CI for each coefficient and quantile
```

```r
ci_results <- list()

for (j in 1:length(quantiles)) {
  tau <- quantiles[j]
  ci_list <- list()

  for (var in dimnames(boot_coefs)[[3]]) {
    boot_vals <- boot_coefs[, j, var]
    orig_val <- orig_coefs[, j][var]

    # Percentile CI
    perc_ci <- quantile(boot_vals, probs = c(0.025, 0.975))

    # Basic CI: 2*theta_hat - quantile
    basic_ci <- 2 * orig_val - rev(perc_ci)

    ci_list[[var]] <- list(
      percentile = perc_ci,
      basic = basic_ci,
      estimate = orig_val
    )
  }
  ci_results[[paste0("Q", tau)]] <- ci_list
}

var_names <- names(ci_results[[1]])

# Create a list to store tables for each quantile
summary_tables <- list()

# Loop through each quantile
for (q in names(ci_results)) {
  coef_info <- ci_results[[q]]

  # For each variable, extract estimate, percentile CI, and basic CI
  table_q <- data.frame(
    Variable = var_names,
    Estimate = sapply(var_names, function(v) coef_info[[v]]$estimate),
    Perc_Lower = sapply(var_names, function(v) coef_info[[v]]$percentile[1]),
    Perc_Upper = sapply(var_names, function(v) coef_info[[v]]$percentile[2]),
    Basic_Lower = sapply(var_names, function(v) coef_info[[v]]$basic[1]),
    Basic_Upper = sapply(var_names, function(v) coef_info[[v]]$basic[2])
  )

  summary_tables[[q]] <- table_q
}

(summary_tables$Q0.1)[-1]
```

```
##                            Estimate   Perc_Lower Perc_Upper  Basic_Lower
## (Intercept).(Intercept)  8.909051840  -0.30972022 20.5908064  -2.77270270
## rm.rm                    3.640920418   1.74023331  4.9419532   2.33988766
## lstat.lstat             -0.466030533  -0.56553967 -0.3906721  -0.54138893
## chas.chas                3.357883765   0.88620691  5.7898618   0.92590574
```

```
## nox.nox                 -9.830145408 -13.88737095 -0.8233528 -18.83693805
## tax.tax                 -0.007864568  -0.01367398 -0.0044304  -0.01129874
##                          Basic_Upper
## (Intercept).(Intercept) 18.127823903
## rm.rm                    5.541607521
## lstat.lstat             -0.366521397
## chas.chas                5.829560621
## nox.nox                 -5.772919871
## tax.tax                 -0.002055161
```

```
summary_tables$Q0.25[-1]
```

```
##                            Estimate  Perc_Lower  Perc_Upper Basic_Lower
## (Intercept).(Intercept)  4.475169522 -5.71260940 12.70389393 -3.75355488
## rm.rm                    4.090566739  2.84239404  5.66859418  2.51253929
## lstat.lstat             -0.474525225 -0.60177929 -0.36793657 -0.58111388
## chas.chas                3.924562898  1.55514408  5.31436333  2.53476246
## nox.nox                 -3.019409290 -8.45047276  3.18476754 -9.22358612
## tax.tax                 -0.009107052 -0.01474603 -0.00529281 -0.01292129
##                          Basic_Upper
## (Intercept).(Intercept) 14.662948446
## rm.rm                    5.338739433
## lstat.lstat             -0.347271164
## chas.chas                6.293981715
## nox.nox                  2.411654177
## tax.tax                 -0.003468072
```

```
summary_tables$Q0.5[-1]
```

```
##                            Estimate   Perc_Lower  Perc_Upper  Basic_Lower
## (Intercept).(Intercept) -10.01722415 -17.70680771  4.23934680 -24.27379509
## rm.rm                     6.44164789   4.21741949  7.51313670   5.37015908
## lstat.lstat              -0.40267652  -0.58028908 -0.30854589  -0.49680714
## chas.chas                 2.86183734   1.40511269  4.07600052   1.64767416
## nox.nox                   1.61188114  -3.45918267  8.08528698  -4.86152471
## tax.tax                  -0.01213562  -0.01523945 -0.00647902  -0.01779222
##                           Basic_Upper
## (Intercept).(Intercept) -2.327640586
## rm.rm                    8.665876289
## lstat.lstat             -0.225063954
## chas.chas                4.318561992
## nox.nox                  6.682944947
## tax.tax                 -0.009031795
```

```
summary_tables$Q0.75[-1]
```

```
##                            Estimate   Perc_Lower   Perc_Upper  Basic_Lower
## (Intercept).(Intercept) -22.727554019 -32.58719526 -5.248372593 -40.20673544
## rm.rm                     8.062252639   5.67929946  9.520467100   6.60403818
## lstat.lstat              -0.308574111  -0.57880360 -0.153929302  -0.46321892
## chas.chas                 4.845680785   0.51522251  8.595695381   1.09566619
## nox.nox                   5.776890734  -2.42335654 14.063617920  -2.50983645
## tax.tax                  -0.006540911  -0.01172194 -0.003076613  -0.01000521
##                           Basic_Upper
## (Intercept).(Intercept) -12.867912775
## rm.rm                    10.445205817
```

```
## lstat.lstat              -0.038344626
## chas.chas                 9.176139060
## nox.nox                  13.977138007
## tax.tax                  -0.001359879
```

```
summary_tables$Q0.9[-1]
```

```
##                             Estimate   Perc_Lower   Perc_Upper Basic_Lower
## (Intercept).(Intercept) -27.84486562 -41.73415705 -1.267019977 -54.4227113
## rm.rm                     8.53462569   4.86709562 10.322999103   6.7462523
## lstat.lstat              -0.34372958  -0.82581196 -0.046274580  -0.6411846
## chas.chas                 6.21314747   1.13158305 17.944068529  -5.5177736
## nox.nox                  13.54088900   0.94689653 29.357833164  -2.2760552
## tax.tax                  -0.00167592  -0.01193433  0.009376756  -0.0127286
##                             Basic_Upper
## (Intercept).(Intercept) -13.955574197
## rm.rm                    12.202155765
## lstat.lstat               0.138352803
## chas.chas                11.294711882
## nox.nox                  26.134881468
## tax.tax                   0.008582488
```

Problem 2.1.2

Modify your bootstrap code to run in parallel using *foreach* + *%dopar%* or *mclapply* in R, report the time the runs with and without parallelization.

```
library(foreach)
library(doParallel)
```

```
## Loading required package: iterators
```

```
## Loading required package: parallel
```

```
library(parallel)
library(abind)

set.seed(2)

boot_coefs_serial <- list()
B <- 1000

time_serial <- system.time({
  for (b in 1:B) {
    idx <- sample(1:nrow(Boston), replace = TRUE)
    boot_data <- Boston[idx, ]

    boot_coefs_serial[[b]] <- sapply(quantiles, function(tau) {
      model <- rq(medv ~ rm + lstat + chas + nox + tax, data = boot_data, tau = tau)
      coef(model)
    })
  }
})
# Register cores
num_cores <- parallel::detectCores() - 1
cl <- makeCluster(num_cores)
registerDoParallel(cl)
```

```
# Start timing
time_parallel <- system.time({
  boot_coefs_parallel <- foreach(b = 1:1000, .combine = "rbind", .packages = "quantreg") %dopar% {
    idx <- sample(1:nrow(Boston), replace = TRUE)
    boot_data <- Boston[idx, ]
    sapply(quantiles, function(tau) {
      model <- rq(medv ~ rm + lstat + chas + nox + tax, data = boot_data, tau = tau)
      coef(model)
    })
  }
})

stopCluster(cl)  # Always stop the cluster

time_mclapply <- system.time({
  boot_coefs_mcl <- do.call(rbind, mclapply(1:1000, function(b) {
    idx <- sample(1:nrow(Boston), replace = TRUE)
    boot_data <- Boston[idx, ]
    sapply(quantiles, function(tau) {
      model <- rq(medv ~ rm + lstat + chas + nox + tax, data = boot_data, tau = tau)
      coef(model)
    })
  }, mc.cores = num_cores))
})

print(time_serial)
```

```
##    user  system elapsed
##   3.393   0.127   3.590
```

```
print(time_parallel)
```

```
##    user  system elapsed
##   0.177   0.151   2.369
```

```
print(time_mclapply)
```

```
##    user  system elapsed
##   3.486   1.355   1.839
```

The runtime without parallelization is higher than the runtimes of both parallelizations method using foreach and mclapply. mcapply is the fastest in real world time but its total GPU time is the highest.

Problem 2.2 Penalized Linear regression

Problem 2.2.1

Randomly select 80% data (as training data) to fit a linear regression for *medv* and treat all other variables as potential predictors. Implement a lasso (L1-penalized) linear regression to model medv as a function of these variables. Use 5-fold cross-validation to select the optimal turning parameter λ. Report the cross-validation errors across the grid of λs and identify the optimal λ that minimizes the average validation error.

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```r
library(glmnet)
```

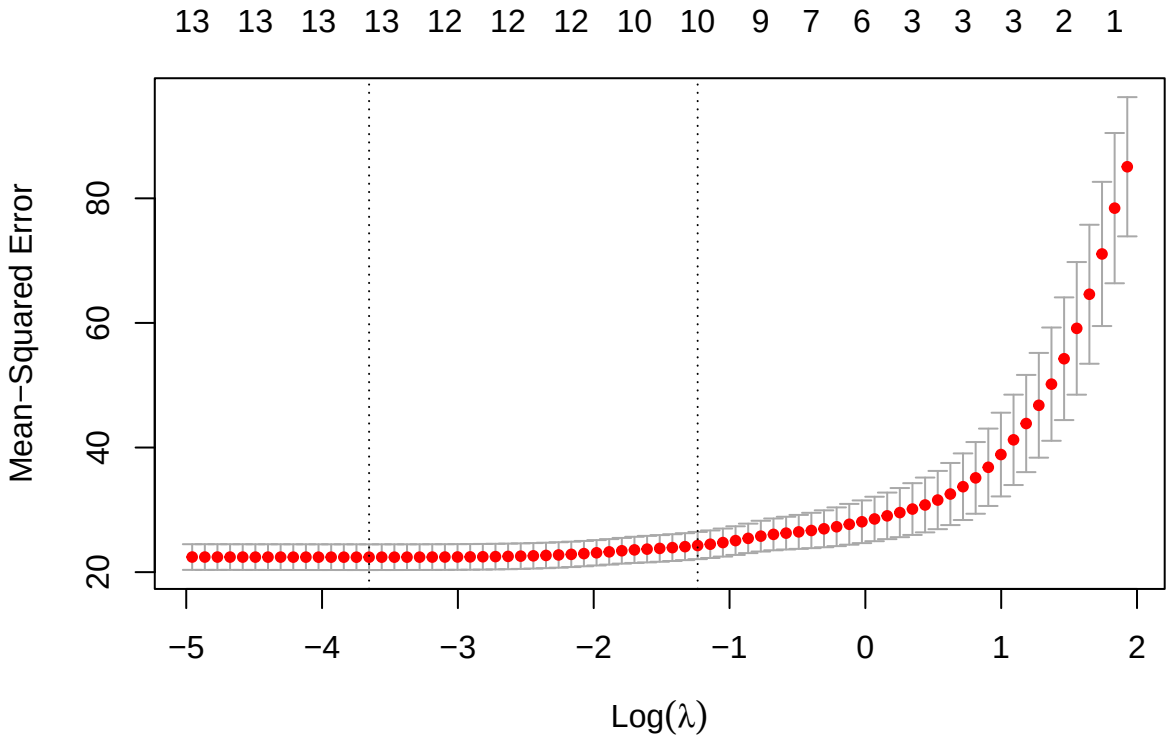
```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following object is masked from 'package:SparseM':
##
##     det
```

```
## Loaded glmnet 4.1-8
```

```r
# Split data
set.seed(2)
train_index <- createDataPartition(Boston$medv, p = 0.8, list = FALSE)
train_dat <- Boston[train_index, ]
test_dat <- Boston[-train_index, ]

train_x <- model.matrix(medv ~ . - 1, data = train_dat)
train_y <- train_dat$medv

model.lasso <- cv.glmnet(train_x, train_y, alpha = 1, nfolds = 5)
plot(model.lasso)
```




```r
cv_results <- data.frame(
  lambda = model.lasso$lambda,
  cv_error = model.lasso$cvm
)

head(cv_results)
```

```
##     lambda cv_error
```

```
## 1 6.877867 85.06698
## 2 6.266856 78.42523
## 3 5.710126 71.07210
## 4 5.202854 64.60329
## 5 4.740647 59.12844
## 6 4.319501 54.25530
```

```
best_lambda <- model.lasso$lambda.min
cv_results[cv_results$lambda == best_lambda, ]
```

```
##        lambda cv_error
## 61 0.02589473 22.40347
```

The optimal tuning parameter λ is 0.0258947. And its corresponding CV error is 22.4034714. By taking natural log of the optimal λ, the result aligns with the above plot.

Problem 2.2.1

Using that selected λ, re-fit the lasso model to the full dataset. Record selected variables under this final model. Fit the training data with the selected variables, and use it to predict the housing prices in the remaining 20% testing data

```
full_x <- model.matrix(medv ~ . - 1, data = Boston)
full_y <- Boston$medv

final_lasso <- glmnet(full_x, full_y, alpha = 1, lambda = best_lambda)
coef_lasso <- coef(final_lasso)
selected_vars <- rownames(coef_lasso)[which(coef_lasso != 0)]
selected_vars
```

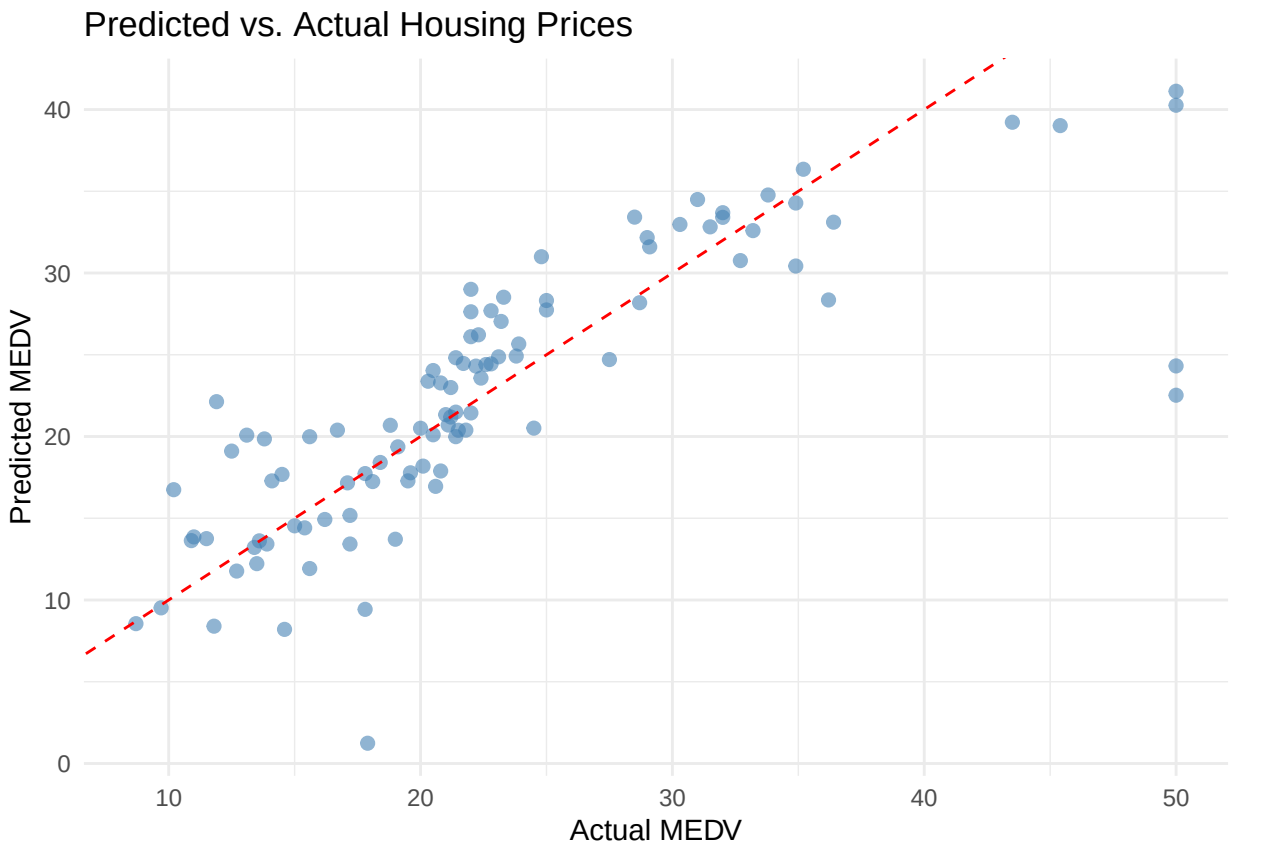
```
##  [1] "(Intercept)" "crim"        "zn"          "chas"        "nox"
##  [6] "rm"          "dis"         "rad"         "tax"         "ptratio"
## [11] "black"       "lstat"
```

```
formula_str <- paste("medv ~", paste(selected_vars[-1], collapse = " + "))
final_formula <- as.formula(formula_str)
final_lm <- lm(final_formula, data = train_dat)
pred_test <- predict(final_lm, newdata = test_dat)
test_y <- test_dat$medv

# Evaluate prediction performance
test_mse <- mean((pred_test - test_y)^2)
test_rmse <- sqrt(test_mse)

cat("Test RMSE:", test_rmse, "\n")
```

```
## Test RMSE: 5.507516
```

```
results_df <- data.frame(
  Actual = test_y,
  Predicted = pred_test
)

# Scatter plot with reference line
ggplot(results_df, aes(x = Actual, y = Predicted)) +
  geom_point(color = "steelblue", alpha = 0.6, size = 2) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
```

```
  labs(
    title = "Predicted vs. Actual Housing Prices",
    x = "Actual MEDV",
    y = "Predicted MEDV"
  ) +
  theme_minimal()
```



By using the selected variables, we fit model and make prediction on the test data with RMSE = 5.5075163. The plot also implies how well the predictions are. We observe most of the predicted (blue) points are close to the red dashed line (where actual MEDV equals to predicted MEDV).

Problem 2.2.2

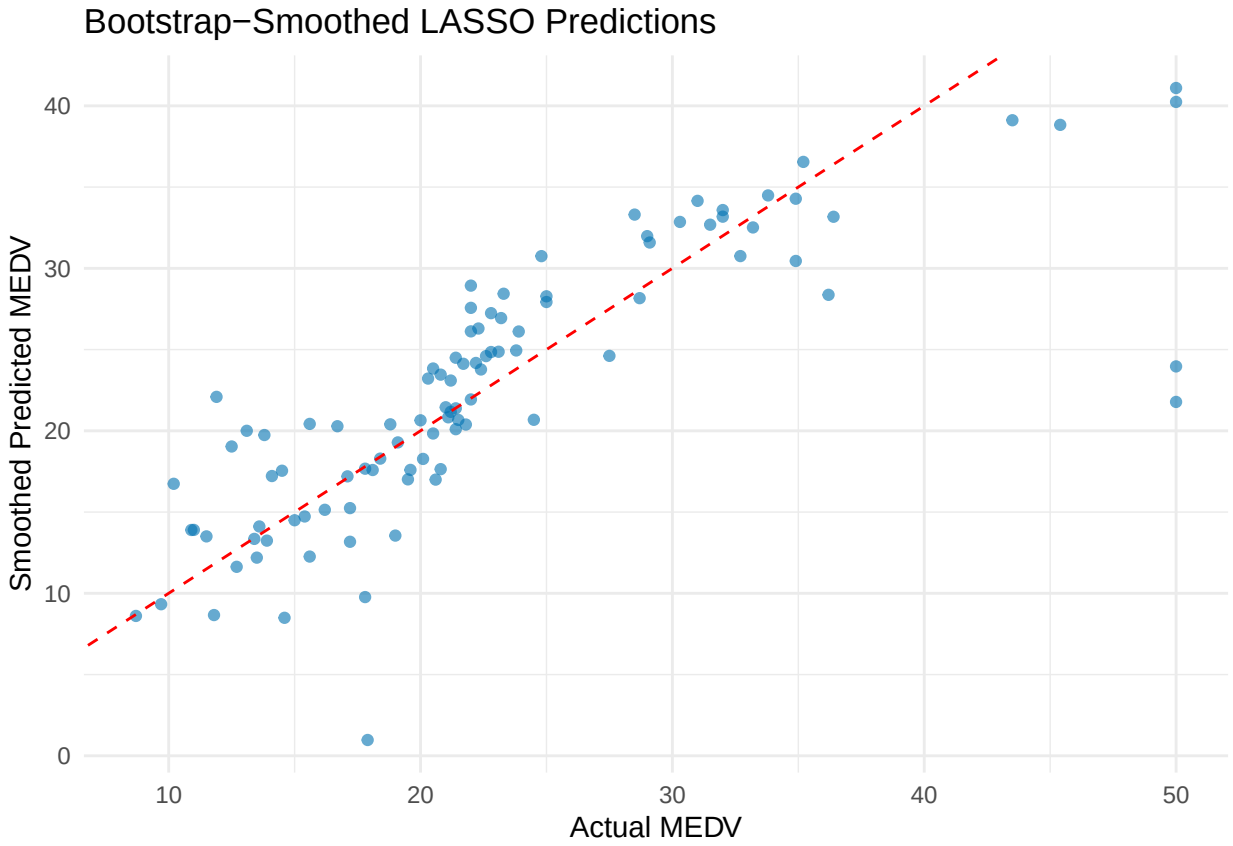
Following the Efron's bootstrap-smoothing idea, bootstrap the data, use 5-fold cross-validation to obtain a new set of selected variables and coefficient estimates for each bootstrap sample; $B=500%. Use each bootstrap model to predict the housing price in the reaming 20% testing data, and average them as the final predict.

```
test_x <- model.matrix(medv ~ . - 1, data = test_dat)

B <- 500
pred_matrix <- matrix(NA, nrow = nrow(test_dat), ncol = B)

for (b in 1:B) {
  boot_idx <- sample(1:nrow(train_dat), replace = TRUE)
  boot_dat <- train_dat[boot_idx, ]

  boot_x <- model.matrix(medv ~ . - 1, data = boot_dat)
```

```
  boot_y <- boot_dat$medv

  cv_boot <- cv.glmnet(boot_x, boot_y, alpha = 1, nfolds = 5)

  # Final model using lambda.min
  final_boot <- glmnet(boot_x, boot_y, alpha = 1, lambda = cv_boot$lambda.min)

  # Predict on the test set
  pred_matrix[, b] <- predict(final_boot, newx = test_x)
}

# Average the predictions as final predicts
final_pred <- rowMeans(pred_matrix)

actual_test <- test_dat$medv
rmse_smooth <- sqrt(mean((final_pred - actual_test)^2))
cat("Bootstrapped Smoothed RMSE on test set:", round(rmse_smooth, 4), "\n")
```

```
## Bootstrapped Smoothed RMSE on test set: 5.5479
```

```
# Visualization
df_results2 <- data.frame(
  Actual = actual_test,
  Predicted = final_pred
)

ggplot(df_results2, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6, color = "#0072B2") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  labs(title = "Bootstrap-Smoothed LASSO Predictions",
       x = "Actual MEDV", y = "Smoothed Predicted MEDV") +
  theme_minimal()
```



For both the evaluations(RMSE) and the visualization, the model's performance on prediction with bootstrap-smoothing idea are similar with the regular 5-fold CV's result. Further comparison will be provided in following section.

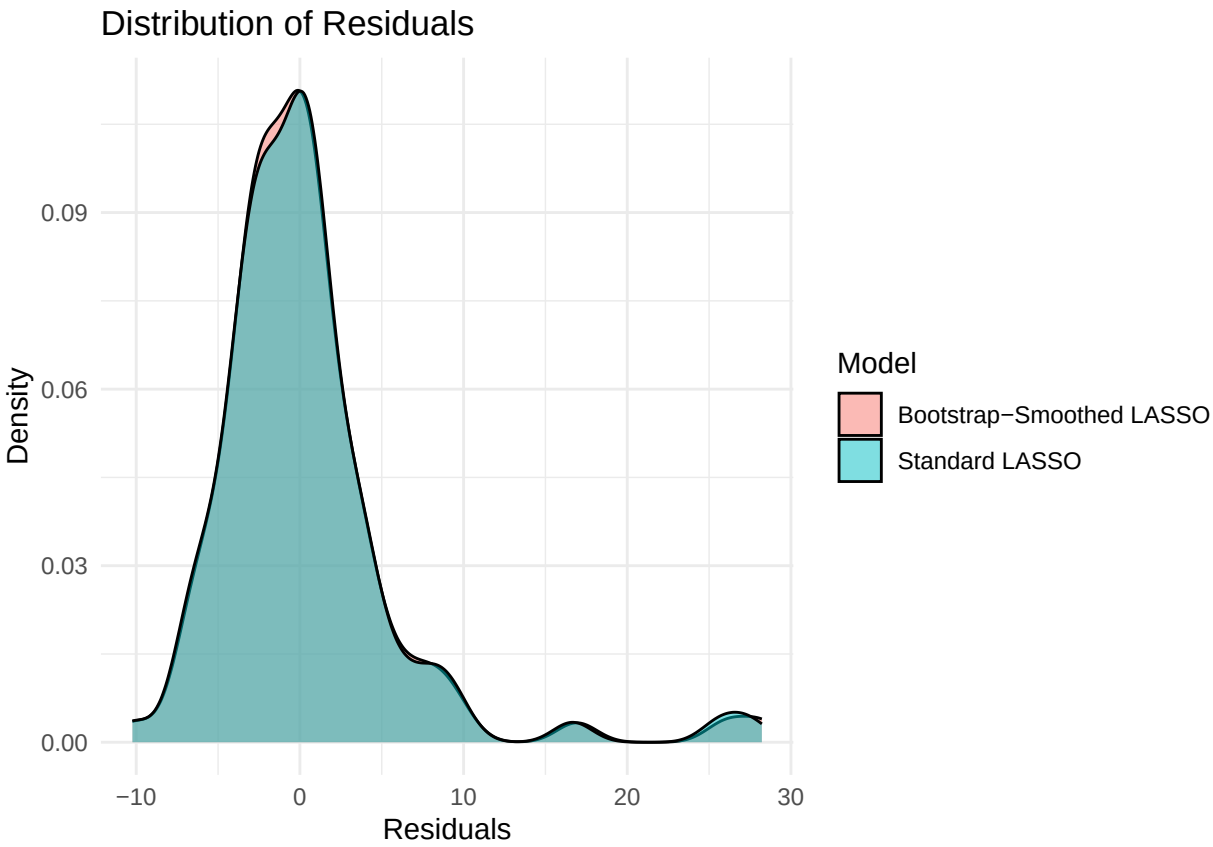
2.2.3 Compare the prediction performance in 2.2.1 and 2.2.2, what do you observe?

```
# R-squared
sst <- sum((test_y - mean(test_y))^2)
ssr_lasso <- sum((pred_test - test_y)^2)
ssr_smooth <- sum((final_pred - test_y)^2)

rsq_lasso <- 1 - ssr_lasso / sst
rsq_smooth <- 1 - ssr_smooth / sst

comparison_df <- data.frame(
  Model = c("Standard LASSO", "Bootstrap-Smoothed LASSO"),
  RMSE = c(test_rmse, rmse_smooth),
  R_squared = c(rsq_lasso, rsq_smooth)
)

print(comparison_df)
```

```
##                       Model     RMSE R_squared
## 1           Standard LASSO 5.507516 0.6402830
## 2 Bootstrap-Smoothed LASSO 5.547858 0.6349939
```

```
resid_lasso <- test_y - pred_test
resid_smooth <- test_y - final_pred
```

```r
res_df <- data.frame(
  Residual = c(resid_lasso, resid_smooth),
  Model = rep(c("Standard LASSO", "Bootstrap-Smoothed LASSO"), each = length(test_y))
)

ggplot(res_df, aes(x = Residual, fill = Model)) +
  geom_density(alpha = 0.5) +
  labs(title = "Distribution of Residuals", x = "Residuals", y = "Density") +
  theme_minimal()
```



By comparing metrices like RMSE, R-squared, and visualization of residuals, we can conclude that both models are good fit and perform well on the prediction task. Bootstrap-smoothed lasso does not significantly imporve the model fit and prediction due to its lower R-sqaured and higher RMSE than standard 5-fold CV lasso. But overall, two models are very similar with each other.